CMP_SC 4650 Digital Image Processing
HW1A: Point Processes in Python
Ellie Nash
9/5/2024

Abstract

Given an RGB image of bone cells, I was tasked with applying point operations to make it more readily analyzable. The primary goal was to extract the bone cell nuclei from the surrounding matter. As the nuclei are differentiated most by their color, I had trouble making meaningful progress using a grayscale version of the image. Using a threshold function on the blue channel allowed optimal segmentation from the rest of the image

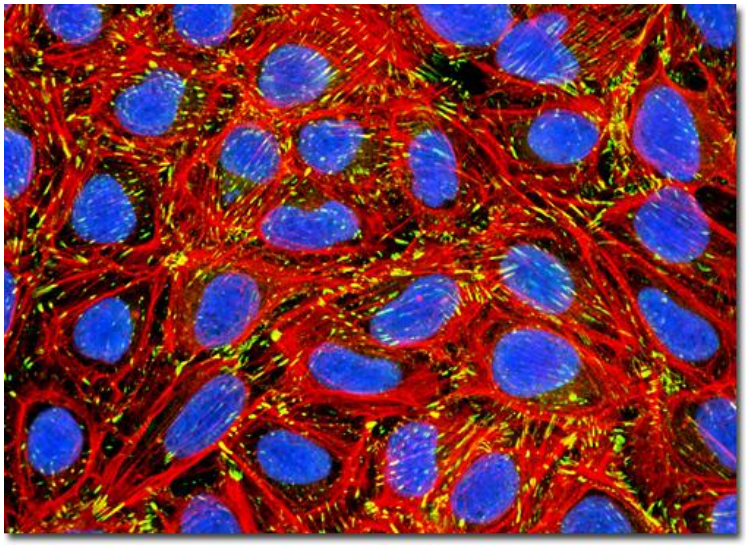
Introduction

Throughout this assignment, I hope to gain a rudimentary understanding of OpenCV Python library. The three main experiments to achieve this will all be performed on the same image of bone cells. I will first convert the image to grayscale, then perform binarization on the resulting image, then compare those results to binarizations based on the blue color channel. To gain the best understanding of image data structures, I plan on using OpenCV functions exclusively for opening and saving files.

Since I used Python natively on a Windows computer, all programs were called in the format of “py rgb2gray.py [Input Image] [Output Image] [Threshold Value, if appropriate].”

Experiments

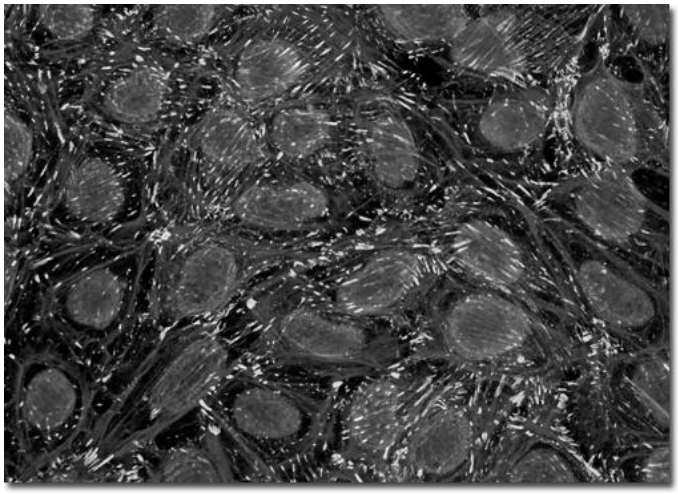
1. Color to Grayscale Conversion



The original RGB image

The conversion to grayscale was performed using a weighted average of the three color channels. OpenCV stores images in a three-dimensional array of 8-byte integers, representing row, column, and color intensity. Using .shape(), I was able to extract the dimensions of the image and create an identically sized numpy array with one color channel. I then created two nested for loops to iterate through each pixel in the image and calculated the gray value using the equation:

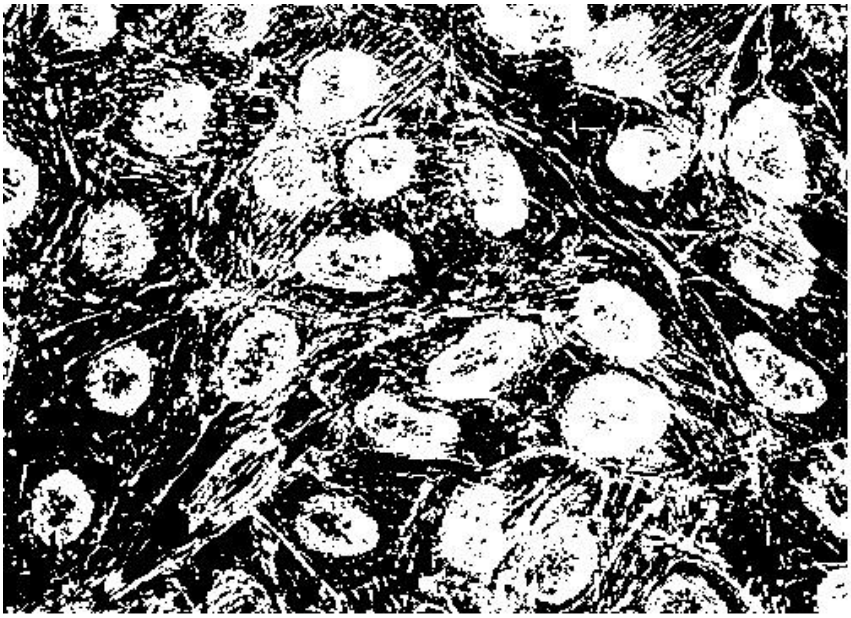
```
imgGrey[y][x] = int(.1140 * img[y][x][0] + .5871 * img[y][x][1] + .2989 *
                                    img[y][x][2])
```



The resulting grayscale image (which is 1/3 the size of the original, perhaps unsurprisingly)

2. **Grayscale Binarization**

The second experiment involved binarizing the grayscale image to segment the nuclei. A threshold value is provided when calling the program, and any pixel with an intensity below it is set to 0. Any above is set to 255. I used the same double-loop structure to iterate through the pixels, with a simple comparison between its intensity and the threshold.

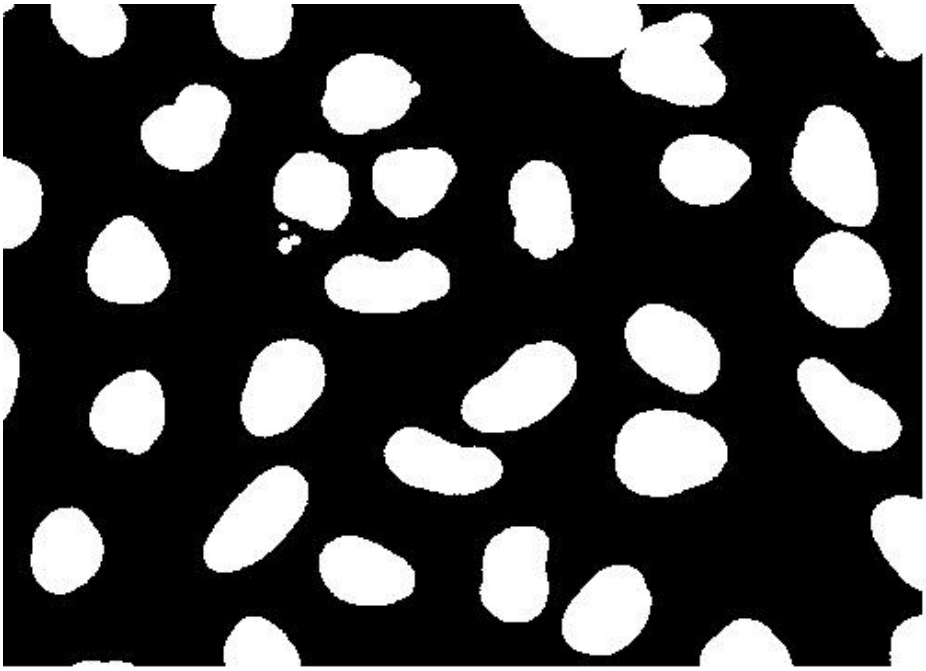


The binarized image with a threshold of 65

As the nuclei are primarily differentiated by their color, this experiment was largely unsuccessful. The conversion to grayscale removed most of the information that distinguished the nuclei. Through a number of iterations, I found that a threshold value of 65 got the closest to segmenting them. Any lower, and too much of the surrounding cells are retained to identify the boundaries of the nuclei. Any higher, and the internal areas of the nuclei would fall below the threshold.

3. **Color Binarization**

The final experiment repeated the process of binarization on the blue channel of the original RGB image of bone cells. The program structure is fairly similar to the previous experiment, except the input image now has three color channels. As such, the comparison to the threshold is performed only on the first index of each pixel (i.e. img[y][x][0]) where the blue intensity is stored. Depending on the result, all three channels must be set to either 0 or 255.



The binarized image with a threshold of 30

Since the blue intensity of the non-nuclei parts of the cells was low, the image could be segmented with a low threshold. I found that a blue intensity value of 30 successfully filtered out the other parts of the cell while retaining the full area of the nuclei.

Conclusion

The programs performed essentially as expected. Since the nuclei are distinguished primarily by their blue color, segmentation using point operations became impossible after the conversion to grayscale. This is particularly true with the conversion equation I used, where the blue channel's weight was insignificant. Performing the segmentation with the blue channel gave nearly perfect results.

I had to rewrite these programs a few different times, mostly to challenge myself by removing library functions. Directly interacting with the image's data structure gave me a much better understanding of how transformations work.